Information Retrieval Research - (2026) 1-20 Submitted 10/26; Published -

A Score Ceiling in Reciprocal Rank Fusion Detects Zero Overlap at Depth $K \leq k$: an Incident and Six BEIR Collections

Benjamin Meadows BEN@BENJAMINMEADOWS.COM
Independent Researcher
ORCID 0009-0007-9698-0910

Editor: -

Abstract

A hybrid search system fuses a keyword retriever and a vector retriever with reciprocal rank fusion (RRF). In the system described here, the keyword query builder was unchanged from the system's first version until 2026-08-18; replayed on ten real queries, it returned nothing for every conversational one, and the result lists had never looked empty because the vector retriever always filled them. The failure was visible in the fused scores. RRF credits a document $1/(k + r)$ from each retriever that ranks it at position r, so with per-retriever depth $K \leq k$ a document found by two retrievers scores above $1/k$ and a document found by one scores at most $1/k$. A result list whose maximum fused score is $1/k$ therefore contains no document that both retrievers returned. That is a statement about overlap, not about either retriever: two nonempty lists can be disjoint, and in the incident four of ten queries had a one- to three-document keyword list that shared nothing with the vector list. I give the derivation and its limits, the bug (quoting every token of an SQLite FTS5 query, which FTS5 reads as an implicit AND), before-and-after aggregates from the production system, and a reproduction on six BEIR collections with three encoders, 22 362 hybrid query runs. The buggy builder returns nothing for 50.8 to 98.0% of queries on five collections, in the order of their content words per query, and the ceiling missed none of those empty legs. Where the two instruments disagree, the ceiling fired over a nonempty leg that shared nothing with the vector leg in 174 runs, mostly short lists on large corpora, and in two runs over lists that shared one document deep in both, the case depth $K > k$ leaves open. On the sixth collection, whose queries are themselves corpus documents, the buggy leg is never empty and almost never relevant: it returns the query's own text, the vector leg agrees, and the ceiling never fires. Repairing the leg raises overlap in every collection–encoder pair; at nDCG@10 the repaired hybrid is above the vector retriever alone in 5 of 18 pairs (2 of them significant under a paired randomization test with Holm's correction) and below it in 13 (5 significant). Code, results, per-query diagnostics and this paper's source are at `https://github.com/TheBenMeadows/dead-retriever`.

Keywords: reciprocal rank fusion, hybrid retrieval, rank fusion diagnostics, SQLite FTS5, BM25, dense retrieval

1 Introduction

The system is a personal search engine over about 30 000 to 217 000 text chunks per corpus, split across five SQLite databases that share one retrieval module. Each query runs two retrievers over the same chunks: an SQLite FTS5 index ranked by BM25 (Robertson and

©2026 held by the author(s) License: CC-BY 4.0

Zaragoza, 2009; Hipp et al., 2026), and a 768-dimensional embedding index searched by exact cosine. The two ranked lists, each cut at depth $K = 60$, are fused with reciprocal rank fusion at $k = 60$ (Cormack et al., 2009).

On 2026-08-07 a recency-decay feature was evaluated and rejected because RRF scores were "flat near the top": adjacent ranks differed by about 1.4%, so even a 5% age multiplier acted as a strong reranker rather than a tie-break. That flatness was recorded as a property of fusion. On 2026-08-18, while investigating why the keyword side rarely seemed to matter, I logged the keyword retriever's row counts for ten real queries. Five of five conversational queries matched zero chunks, as did a two-word place-name query. Only queries of two or three technical tokens matched anything. The query builder had been the same line of code since the system was built, and all four sibling engines carried it; that it had behaved this way on prose queries throughout is an inference from the unchanged code and the replay, since the system keeps no query log.

The fused scores had said so from the start. Over the fused lists of those ten queries (at most 120 documents each at $K = 60$, fewer after per-file deduplication), the maximum score was exactly $1/60 = 0.01\overline{6}$. The values were $1/60, 1/61, 1/62, \ldots$ with gaps where deduplication removed a chunk: every fused score was a single term $1/(k + r)$, so no document had been credited by both retrievers. The "1.4% flatness" was misread twice. Between consecutive singleton scores it is the relative step of the formula, $\big(1/(k+r) - 1/(k+r+1)\big)/\big(1/(k+r)\big) = 1/(k+r+1)$, 1.6% at the top of the list and 1.4% around rank 10, and two live lists that agreed on every document would show the same relative step at twice the absolute scale. What marked the incident was not the step but the scale: nothing scored above 1/60.

This paper makes four contributions.

1. A derivation (Section 3): with per-retriever depth $K \leq k$, the maximum RRF score over a result list is at most $1/k$ if and only if no document in the list was returned by more than one retriever; one direction survives at any depth, the other needs $K \leq k$, and the failure mode at $K > k$ is characterised ($r_1 r_2 \geq k^2$).

2. The incident (Sections 4 and 5): the query builder that produced an implicit FTS5 AND, the fix, and aggregate before-and-after measurements from the production system.

3. A reproduction on six BEIR collections and three encoders, 22 362 hybrid query runs (Sections 6 and 7): how often the buggy builder returns nothing, how the ceiling rate tracks the empty-leg rate, where the two disagree and why, one collection on which the test is blind by construction, and what repairing the leg did to effectiveness, with paired significance tests.

4. Operator guidance (Section 8) on what to log and how to read it, bounded by Section 9.

Section 2 places the ceiling among the fusion and monitoring literature. Two negative results measured on the production system beside the dead leg are in Appendix A; they are part of the incident's record, not evidence about the ceiling.

2 Background and related work

Reciprocal rank fusion was proposed by Cormack et al. (2009), who set $k = 60$ by experiment; the system here used that constant unchanged, which is what made the 1/60 ceiling recognisable on sight. The older fusion literature studied overlap between the lists being fused: CombSUM sums the scores a document receives and CombMNZ additionally multiplies by the number of systems that returned it (Fox and Shaw, 1994), so that CombMNZ rewards multi-system agreement explicitly; Lee (1997) showed that fusion helps when the systems agree on relevant documents more than on non-relevant ones, and Montague and Aslam (2001) treated the score-scale problem that rank fusion sidesteps. Rank-biased overlap (Webber et al., 2010) is the principled measure of agreement between two indefinite rankings; the both-leg count in Section 7 is its crudest relative, measured where it had gone to zero. On hybrid lexical and dense retrieval, Luan et al. (2021) and Gao et al. (2021) establish the complementarity the hybrid relies on, and Bruch et al. (2024) analyse fusion functions and show when RRF is and is not competitive with a tuned convex combination; this paper is about a failure upstream of that choice, where one of the lists is empty. Query performance prediction (Cronen-Townsend et al., 2002) asks, per query, whether a retriever did well; the ceiling asks a cruder question of the fusion, whether its inputs agreed on anything, and the instrumentation it calls for is the kind Cooper (1998) argued retrieval systems should carry. Kamphuis et al. (2020) show how much BM25 scores vary across implementations of the same formula; the FTS5 variant here is one more, and its query grammar, not its scoring, is what failed.

What the ceiling adds to the two obvious instruments is cost and placement, not information. Per-retriever row counts need a counter in each retriever's wrapper, and a direct overlap count needs both id lists retained past fusion; the maximum fused score is one number the fusion already computes, available wherever the fused list is, including in a log that recorded only scores. The reproduction measures how far that one number can be trusted as a stand-in for the other two. ArguAna (Wachsmuth et al., 2018), one of the six collections, is built so that each query is itself a corpus document; BEIR's evaluator removes a document whose id equals the query's id before scoring for that reason (Thakur et al., 2021), and Section 7 shows what the construction does to a keyword leg that requires every query token.

3 The ceiling, and what it does and does not say

Let L retrievers each return a ranked list cut at depth K in which every document appears once, let k be a positive integer, and let $r_\ell(d) \in \{0, \ldots, K-1\}$ be document d's 0-based rank in retriever ℓ's list when present. Reciprocal rank fusion scores

$$s(d) \;=\; \sum_{\ell : d \in \text{list}_\ell} \frac{1}{k + r_\ell(d)}.$$

A document present in exactly one list scores at most $1/k$ (rank 0). A document present in two lists scores at least $2/(k + K - 1)$ (last place in both). The second bound exceeds the first whenever

$$\frac{2}{k + K - 1} > \frac{1}{k} \iff K \leq k,$$

so with $K \leq k$ (here $K = k = 60$), every document found by two or more retrievers outscores every document found by one. Under that setting the following holds exactly, in both directions, for any list with at least one result:

> $\max_d s(d) \leq 1/k$ if and only if no document in the list was returned by more than one retriever.

Three qualifications bound what that means in practice. First, it is a statement about overlap. Two retrievers that both return K documents and share none of them produce the same ceiling as one retriever and one empty list, and so does a keyword leg that returns one or two documents the vector leg did not; with more retrievers, any number of pairwise-disjoint live lists do too. In the incident, the keyword leg returned rows on four of the ten queries and the ceiling held on all ten (Table 1); those four were lists of one to three chunks, with nothing in common with the vector top 60. The ceiling says no document was reinforced by agreement; the per-retriever row counts say whether that is because a leg is empty, nearly empty, or live and disjoint, and a system that fuses should log both. Second, the equivalence needs $K \leq k$. One direction survives at any depth: a maximum above $1/k$ proves that some document was returned by two retrievers, since a singleton scores at most $1/k$ for every K. The other does not. With $K > k$, a document ranked deep in both lists scores below $1/k$ (rank 90 in both at $k = 60$ gives $2/150$), and one ranked exactly k in both scores exactly $1/k$; two ranks r_1, r_2 combine to more than $1/k$ exactly when $r_1 r_2 < k^2$. So at $K > k$ a maximum of $1/k$ shows that every retriever's rank-0 document is exclusive and leaves deeper overlap open, and a system that keeps per-retriever ranks should count shared documents directly. Section 7, which uses $K = 100 > k$, reports both and measures how often the two disagree. Third, with two retrievers an empty leg always produces the ceiling when the other leg returned anything, so the test cannot miss one and everything it gets wrong is in the other direction; with three or more, an empty leg can hide behind two live legs that overlap. The equivalence itself holds for any L. The one-based RRF of Cormack et al. (2009) shifts every figure by one: the singleton ceiling is $1/(k+1)$, and strict separation holds for $K \leq k+1$. Throughout, the scores compared are the unweighted RRF sums before any downstream filtering or rescaling, and "at the ceiling" means $\max_d s(d) \leq 1/k + 10^{-12}$.

4 The bug

The keyword query builder was

```
fq = " ".join(f'"{t}"' for t in tokens)
```

which, for the query *how do I fix the empty feed*, produced

```
"how" "do" "i" "fix" "the" "empty" "feed"
```

FTS5 reads a sequence of phrases separated by whitespace as an implicit AND (Hipp et al., 2026, §3): every phrase must occur in the same row. Rows here were 1 400-character chunks, so a conversational query of six or seven tokens almost never matched. Two further details made the constraint stricter than "every token present". The tokeniser kept hyphens, dots and some punctuation inside a token, and FTS5's own tokeniser splits on them, so a token

such as `il-6` became a two-word phrase, an adjacency constraint. And an FTS5 syntax error was caught and ignored, so a malformed query also returned nothing without a trace. Two- or three-token technical queries did match, which is why spot checks during development passed, and the vector retriever returns K documents whenever the corpus has that many, so result lists were always full.

The fix has two parts, and the second is not cosmetic. Tokens are joined with `OR` and ranked by BM25; a raw OR over all tokens matched about 100 000 chunks and cost 161 ms per query, so stopwords and tokens shorter than three characters are dropped first, after which a typical query matches about 3 500 chunks in 2 to 4 ms. If a query is all stopwords, the raw tokens are used. Section 7 measures both variants on public data.

5 Measurements on the production corpus

The corpus is private. This section reports counts, timings and overlaps recorded at the time, with no query or document text; the figures are in `results/production-aggregates.json` in the released repository and are checkable by nobody but the operator, which is the reason the public reproduction exists. Ten real queries were replayed before and after the fix on the same shard (Table 1).

	implicit AND	OR + stopword drop
Queries on which the keyword leg returned any row	4 / 10	10 / 10
Queries with a document found by both legs	0 / 10	10 / 10
Maximum fused score over the fused list	$1/60 = 0.0167$	0.025 – 0.033
Keyword leg latency, conversational queries	–	2 – 4 ms

Table 1: The keyword leg before and after the fix, ten real queries, one production shard, $K = k = 60$. The fused list holds at most 120 documents at this depth. The "before" latency is omitted because a query that matches nothing is fast; a raw OR without the stopword drop took 161 ms. On the four "before" queries where the leg returned rows, it returned one to three chunks, none shared with the vector top 60.

The same builder was present at a different line number in four sibling engines over four other corpora, each inert on prose queries in the same way; the five have since been merged into one module.

6 Experimental protocol

To check that the effect is a property of the query builder and not of one private corpus, six retrieval conditions were run on six BEIR collections (Thakur et al., 2021) with three encoders.

Collections. SciFact (Wadden et al., 2020) (claim sentences against scientific abstracts), NFCorpus (Boteva et al., 2016) (short nutrition titles against medical documents), ArguAna (Wachsmuth et al., 2018) (whole arguments against counter-arguments), SciDocs

(Cohan et al., 2020) (paper titles against abstracts), FiQA (Maia et al., 2018) (finance questions against forum posts) and TREC-COVID (Voorhees et al., 2020) (COVID-19 questions against CORD-19 abstracts); Table 2 gives their sizes. Data came through `ir_datasets` (MacAvaney et al., 2021); only queries with relevance judgements are run. The collections differ in domain, corpus size, document length and relevance style as well as in query length, so contrasts between them are observations, not controlled comparisons.

Encoders. `nomic-embed-text-v1.5` (Nussbaum et al., 2024), the public release of the production encoder, `bge-base-en-v1.5` (Xiao et al., 2023) and `e5-base-v2` (Wang et al., 2022), all 768-dimensional, each with the query and document prefixes its model card prescribes (Appendix C). Documents are embedded as title and text joined by a space; vectors are normalised and searched by exact cosine.

Keyword leg. The builder's tokeniser is the regular expression `[\w$#.@-]+` over the lower-cased query. The buggy builder quotes every token and joins with spaces (implicit AND). The fixed builder drops the 118 stopwords listed in Appendix C and every token under three characters, then joins the rest with `OR`, falling back to the raw tokens when nothing survives; a third condition joins every raw token with `OR`. The FTS5 index holds title and text columns with default BM25 weights.

Conditions. BM25-AND (the buggy builder), BM25-OR-RAW (OR over every token), BM25-OR (the fix), DENSE, HYBRID-AND (dense + buggy keyword, RRF) and HYBRID-OR (dense + fixed keyword, RRF). Depth $K = 100$ for every retriever and $k = 60$ for fusion, so the ceiling is read in its one-sided form and overlap is also counted directly from the ranks. The 18 collection–encoder pairs give $3\,727 \times 3 \times 2 = 22\,362$ hybrid query runs. The three encoders rerun the same queries, so they are not independent replications.

Diagnostics. For each hybrid condition the runner records, per query, whether the keyword leg returned nothing, how many documents it returned, whether the maximum fused score sat at $1/k$, the maximum itself, and how many documents both legs credited. These see the lists as fused.

Effectiveness. nDCG@10 and Recall@100 through `pytrec_eval` (Van Gysel and de Rijke, 2018). Before scoring, a document whose id equals the query's id is removed, the default of BEIR's evaluator; this touches 1 298 of ArguAna's 1 406 queries and 55 of FiQA's 648, and no query elsewhere; on FiQA none of the 55 same-id documents reached a fused top 10 or was relevant, so its figures are the same with and without the rule. The 108 ArguAna queries whose text is a corpus document under another id are not touched. Differences between conditions are tested per pair with a two-sided paired randomization test over queries (sign flips, 10^5 trials; **?**) and Holm's correction within each family of 18 pairs; a difference is called significant at $\alpha = 0.05$ after correction. Per-query scores and the test's output are in `results/`.

Hardware. Every run reported here was made in one Kaggle kernel on an NVIDIA T4 (Appendix C). Query times are mean wall-clock per query for the dense search, the keyword search and the fusion, with query encoding excluded; the keyword search and the fusion run on the host's CPU, which is shared, so times compare conditions within a collection and only loosely across collections.

7 Results

Whether the keyword leg is alive and overlapping with the dense leg, and whether the hybrid retrieves better than its parts, are different questions, and the subsections below keep them apart: Table 2 and Figure 1 are about the leg and the ceiling, Tables 3 and 4 about effectiveness, and every condition for every pair is in Appendix B.

				quoted AND		OR after the drop	
Collection	docs	queries	tokens	kw empty	at ceiling	kw empty	at ceiling
SciFact	5 183	300	12.6	98.0%	98.0%	0.0%	0.0%
NFCorpus	3 633	323	3.3	50.8%	51.4–51.7%	7.7%	7.7–8.7%
ArguAna	8 674	1 406	195.1	0.0%	0.0%	0.0%	0.0%
SciDocs	25 657	1 000	9.4	94.2%	94.7–94.9%	0.0%	0.1%
FiQA	57 638	648	11.1	81.8%	87.2–88.4%	0.0%	0.0–0.3%
TREC-COVID	171 332	50	10.6	74.0%	88.0–90.0%	0.0%	0.0%

Table 2: The keyword leg and the ceiling on six BEIR collections. "tokens" is the mean query length in the builder's own tokens. "kw empty" is the fraction of queries on which the keyword leg returned no document; it does not depend on the encoder. "at ceiling" is the fraction on which the maximum fused score of the hybrid was $1/k$, given as the range over the three encoders where they differ.

7.1 The leg

The quoted-AND builder matched nothing on 294 of SciFact's 300 claim-length queries, 942 of SciDocs's 1 000 titles, 530 of FiQA's 648 questions, 37 of TREC-COVID's 50 questions and 164 of NFCorpus's 323 titles. Mean query length alone does not order these: NFCorpus has the shortest queries and the lowest rate, but TREC-COVID's questions are about as long as FiQA's and SciDocs's and come out lower. Counting only the tokens the fixed builder would keep orders all five, in `results/keyword_facts.json`: 8.7 content tokens per query on SciFact, 7.3 on SciDocs, 6.2 on FiQA, 5.7 on TREC-COVID and 2.5 on NFCorpus, against empty rates of 98.0, 94.2, 81.8, 74.0 and 50.8%. This is close to arithmetic rather than a finding about the collections: every quoted token is one more condition a single document must meet, stopwords are met almost everywhere, and the content words are the conditions that fail. Ordering by raw token count puts SciDocs fourth of the five; counting content words moves it to second, where its empty rate sits. Five collections are too few to call it a law, and the collections also differ in corpus size, document length and vocabulary.

BM25 with the fixed builder is a usable retriever on every collection (Appendix B), and after the fix only NFCorpus has queries that match nothing: 25 title queries (7.7%), none of whose tokens occur in the corpus at all, since the raw OR is empty on the same 25. The stopword drop that made the fix cheap in production is cheaper everywhere and costs little effectiveness: BM25-OR-RAW is 2.2 to 18 times slower (13.6 against 4.1 ms on SciFact, 2.6 against 1.2 on NFCorpus, 905 against 52 on ArguAna, whose queries average 195 tokens) and worse at nDCG@10 by 0.001 to 0.010 on five collections. The exception is FiQA, where

OR over every token is better by 0.009 nDCG@10 and 0.012 Recall@100, at 5.6 times the cost.

7.2 A leg that is never empty and almost never relevant

ArguAna's queries are whole arguments, 195 tokens on average, and the quoted-AND builder never returned nothing for them: it returned one document for 1 402 of the 1 406 queries and two for the other four. The reason is the collection's construction. Every query's text is also a document in the corpus, and a query is the one text guaranteed to contain all of its own tokens: on all 1 406 queries the AND leg returned a document whose text equals the query's after whitespace and case normalisation, under the query's own identifier on 1 298 (at the top of the list on 1 296) and under another identifier on the remaining 108. A query's own document is not one of its relevant documents, and the AND leg returned a relevant document for one query in 1 406.

The ceiling cannot see this. The dense leg also returns that document in its top 100 on every query for every encoder, so the two legs share it, the fused maximum is above $1/k$ on every query, and the ceiling never fired: Table 2 shows 0.0% for every encoder. Fusion with this leg changed nothing measurable: HYBRID-AND and DENSE agree to every decimal the result files carry in both metrics for all three encoders, which is what a dead leg looks like from the outside. The ceiling tests for agreement, and a leg that agrees with the other leg on one useless document passes it. Per-leg row counts do not catch it either, since the leg returns a row on every query; what catches it is the distribution of row counts, one document on 99.7% of queries, which is as much an alarm as zero. ArguAna's construction is unusual, but a corpus that indexes the kind of text it is queried with is not: a search engine over a mailbox, a notes vault or a ticket tracker can receive the text of an indexed item as a query. Whether the self-match is then a failure depends on the task; when finding the source item is the point, it is the answer.

7.3 What fusion did with an empty list

When the keyword leg is empty, the fused list is the dense list. On the 294 SciFact queries with an empty keyword leg, the per-query rows show HYBRID-AND's list is DENSE's list: same document count, maximum score exactly 1/60. Over the whole collection the two conditions differ by at most 0.003 nDCG@10 and 0.000 Recall@100 as printed in Tables 3 and 4 (0.0022 and 0.0000 before rounding), all of it from the six live queries, and on SciDocs, with 942 empty legs, by at most 0.001 in each. Two collections moved more, in opposite directions, and the number of live queries does not predict which: on NFCorpus (159 live queries of 323) HYBRID-AND is above DENSE by 0.008 to 0.014 nDCG@10, and on TREC-COVID (13 live queries of 50) it is below by 0.027 to 0.039.

7.4 The ceiling against the row count

An empty keyword leg puts the maximum fused score at exactly $1/k$ whenever the dense leg returned anything, so the set of empty-leg queries is contained in the set of at-ceiling queries, and the ceiling missed no empty leg on any of the 22 362 hybrid query runs. Everything the two instruments disagree on is the excess, at-ceiling queries whose leg was not empty:

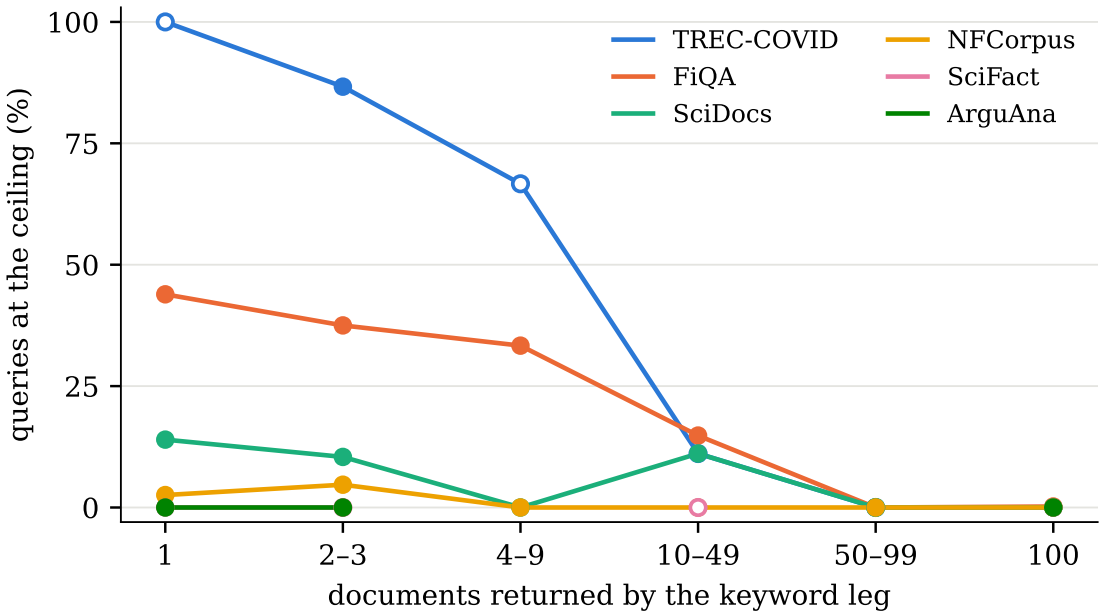


Figure 1: Fraction of hybrid query runs on which the ceiling fired, by the number of documents the keyword leg returned, pooled over both builders and all three encoders. A hollow marker is a cell with fewer than ten runs; a gap is a cell with none. SciFact and ArguAna sit at zero in every cell, under the other lines. Empty legs, at 100% by construction, are not drawn.

176 query runs, 164 under the buggy builder and 12 under the fixed one. On SciFact and ArguAna there was none. Under the buggy builder the excess is 2 or 3 of NFCorpus's 159 live-leg queries (1.3 to 1.9%), 5 to 7 of SciDocs's 58 (8.6 to 12.1%), 35 to 43 of FiQA's 118 (29.7 to 36.4%) and 7 or 8 of TREC-COVID's 13 (53.8 to 61.5%), the range running over the encoders.

In 174 of the 176 the two legs shared no document: read as a zero-overlap test the ceiling was right on them, read as an empty-leg alarm it was wrong, and the row count tells the two readings apart. The other two are the case Section 3 said $K > k$ leaves open. On one FiQA query for bge and one for e5, under the fixed builder, both legs returned 100 documents and shared one, ranked deep enough in both that its fused score did not exceed $1/k$; the ceiling fired with overlap present. Two in 22 362 is rare, but it is not zero, and it is why a system that runs at $K > k$ should count shared documents directly.

How often a live list shares nothing with the dense top K depends on how short it is (Figure 1). Pooled over every collection except ArguAna, where the leg is never disjoint, and over both builders, the ceiling fired on 137 of the 726 live-leg query runs in which the leg returned one to three documents (18.9%), on 35 of 588 with four to 49 (6.0%) and on 4 of 6 636 with 50 or more (0.06%; the last four are two zero-overlap runs and the two overlap-present FiQA runs above). The buggy builder supplies most of the short lists: 612

of the 726 runs in the first bucket, firing on 132 of them, against 5 of 114 for the fixed builder. The short-list rate differs by collection: 0 of 18 on SciFact (5.2k documents), 13 of 330 on NFCorpus (3.6k), 18 of 141 on SciDocs (26k), 90 of 219 on FiQA (58k) and 16 of 18 on TREC-COVID (171k; six distinct queries, each run under three encoders). The four nonzero rates rise with corpus size and SciFact does not fit the ordering, so this is a pattern across four collections that differ in more than size, not a size effect. A short exact-match list over a large corpus was, on these collections, usually a different handful of documents from the dense top 100, whether or not it was any good. The ceiling also fired over a leg of four or more documents on 39 runs, all on FiQA, TREC-COVID and SciDocs, so "a short leg" is not a safe reading of every excess query; the row count is.

7.5 Effectiveness is a separate question

	bge-base-en-v1.5			nomic-embed-text-v1.5			e5-base-v2		
Collection	dense	h-and	h-or	dense	h-and	h-or	dense	h-and	h-or
SciFact	0.740	0.741	0.723	0.703	0.705	0.708	0.719	0.722	0.726
NFCorpus	0.374	0.382	0.363	0.347	0.361*	0.344	0.355	0.368*	0.352
ArguAna	0.641	0.641	0.606*	0.483	0.483	0.517*	0.447	0.447	0.532*
SciDocs	0.217	0.216	0.195*	0.187	0.187	0.182	0.187	0.186	0.183
FiQA	0.406	0.396*	0.348*	0.376	0.367*	0.330*	0.399	0.389*	0.362*
TREC-COVID	0.779	0.744*	0.763	0.823	0.784	0.769	0.696	0.669	0.714

Table 3: nDCG@10 of the dense retriever, the hybrid with the buggy keyword leg (h-and) and the hybrid with the fixed one (h-or). A star marks a hybrid whose paired difference from the dense retriever of the same encoder is significant at $\alpha = 0.05$ after Holm's correction (two-sided randomization test, 10^5 sign flips, 18 pairs per family).

	bge-base-en-v1.5			nomic-embed-text-v1.5			e5-base-v2		
Collection	dense	h-and	h-or	dense	h-and	h-or	dense	h-and	h-or
SciFact	0.967	0.967	0.969	0.928	0.928	0.958	0.963	0.963	0.959
NFCorpus	0.337	0.341*	0.337	0.298	0.303*	0.309	0.319	0.323*	0.328
ArguAna	0.993	0.993	0.991	0.980	0.980	0.984	0.949	0.949	0.981*
SciDocs	0.496	0.495	0.469*	0.404	0.404	0.409	0.421	0.421	0.425
FiQA	0.741	0.740	0.715*	0.682	0.682	0.675	0.732	0.732	0.711
TREC-COVID	0.141	0.140	0.133	0.150	0.148	0.137*	0.128	0.127	0.124

Table 4: Recall@100, same conditions, layout and star as Table 3.

Repairing the leg raises overlap: the mean number of documents credited by both legs rose from at most 4.0 per query (Appendix B) to between 19.0 and 43.7 in every collection–encoder pair. It does not reliably improve retrieval (Tables 3 and 4). Over the 18 pairs, the repaired hybrid is above DENSE at nDCG@10 in 5 and below it in 13, and the paired

test calls 2 of the gains and 5 of the losses significant; at Recall@100 it is above in 9 (one significant; the ninth, NFCorpus with bge, by 0.0006, which the table rounds to a tie) and below in 9 (three significant); and the buggy hybrid is above the repaired one at nDCG@10 in 12 (six significant) and below it in 6 (two significant). The two significant gains over DENSE are on ArguAna, the one collection where BM25 alone (0.484) is above two of the three encoders at nDCG@10, e5 (0.447) and, by 0.001, nomic (0.483): the repaired hybrid gains 0.085 for e5 and 0.034 for nomic, and loses 0.035 for bge, the encoder BM25 does not match, also significantly. The other three gains are small and not significant (0.005 and 0.007 on SciFact for nomic and e5; 0.018 on TREC-COVID for e5). The losses are largest on FiQA, where BM25 is far behind every encoder (0.224 nDCG@10 against 0.376 to 0.406) and the repaired hybrid is 0.037 to 0.058 below DENSE, significant for every encoder.

The buggy hybrid's 12 wins over the repaired one are mostly abstention. In 8 of the 12 the buggy hybrid is not above DENSE: on seven of them its leg was empty on 74 to 94% of queries, on ArguAna with bge it returned only the self-match, so the fused list was the dense list on those queries, and the repaired leg's OR list diluted the top 10. Only on NFCorpus (all three encoders) and SciFact with bge is the buggy hybrid above DENSE as well, by 0.001 on SciFact and by 0.008 to 0.014 on NFCorpus (significant for nomic and e5), where the AND leg was live on half the queries and returned short exact-match lists. Unweighted RRF treats a weak keyword list and a strong dense list alike, which is one reason Bruch et al. (2024) find a tuned convex combination competitive; whether RRF over BM25 and a given dense encoder helps is a question these runs answer only per pair. A monitor for overlap and activity and a verdict on retrieval effectiveness are different instruments.

7.6 How short is too short

To measure the test against short live legs under control, the fixed keyword list was capped at its first n documents and fused with the dense top K, for $K \in \{60, 100\}$, on every SciFact and NFCorpus query where the leg returned anything, for bge and nomic (Table 5). A cap, not a length: a query whose leg returned fewer than n documents keeps its short list in every cell, so each cell measures the ceiling rate given a cap of n over the same 300 and 298 queries. Three things hold in every cell for both encoders. The ceiling fired exactly on the queries where the two lists shared no document: the fired rate and the disjoint rate are equal to the query. The rate falls fast with the cap; the ranges that follow pool the two encoders' cells, of which Table 5 prints bge's: at $K = 60$, a cap of one document leaves the lists disjoint on 8.7 to 15.1% of queries, a cap of three on 2.7 to 6.7%, a cap of ten on 0.7 to 2.3%, and a cap of twenty or more on at most 1.7%, under 1% in 11 of the two encoders' 20 cells. A deeper dense list never raises a figure. The $K > k$ leak, a ceiling with overlap hidden at deep ranks, is impossible by arithmetic unless $(n-1)(K-1) \geq k^2$, which among these cells holds only at $K = 100$ with $n \in \{50, 100\}$; in those eight cells, 2 392 query runs, it did not occur. It did occur twice in the natural FiQA runs above, both with $n = 100$, so the sweep's zero is a property of these two small collections, not of the test. The sweep was not repeated on the four larger collections; the natural row counts above stand in for it there, and they show the short-list rate is several times higher on three of them than these two suggest.

	SciFact		NFCorpus	
n	$K = 60$	$K = 100$	$K = 60$	$K = 100$
1	8.7%	5.3%	15.1%	11.1%
2	5.3%	1.7%	9.1%	6.0%
3	2.7%	1.0%	6.0%	3.4%
5	1.3%	0.7%	3.7%	2.0%
10	1.0%	0.7%	2.3%	1.0%
20	1.0%	0.0%	1.7%	1.0%
50	0.3%	0.0%	1.3%	1.0%
100	–	0.0%	–	1.0%

Table 5: Fraction of live-leg queries on which the ceiling fired when the fixed keyword list is cut to its first n documents and fused with the dense top K (`bge-base-en-v1.5`; 300 SciFact and 298 NFCorpus live-leg queries). In every cell this fraction equals the fraction of queries whose two lists were disjoint, and the fraction that fired with overlap present is zero. The nomic encoder's figures are within 4.4 points of these at $n = 1$ and within 1.0 point from $n = 3$ on; they are in `results/*.sweep.json`.

8 What to check in your own system

Log, per query, each retriever's row count and the maximum fused score. If the maximum sits at $1/k$ (or $1/(k+1)$ for one-based ranks) on more than a few queries, then with per-retriever depth $K \leq k$ the retrievers are not agreeing on anything, and at greater depth no document is shared at ranks r_1, r_2 with $r_1 r_2 < k^2$; read the row counts to learn whether one of them is empty, nearly empty, or retrieving from a different part of the corpus, and at greater depth count shared documents directly. A ceiling over a short live list was common on the larger collections here, so a ceiling is a reason to read the row count, not a verdict. Look at the distribution of row counts, not only at zero: a leg that returns one document on nearly every query is as suspect as one that returns none, and if the corpus indexes the kind of text it is queried with (messages, notes, tickets), check whether that one document is the query itself and whether returning it is the task. Test the keyword builder with a conversational sentence, not a two-word term, and treat zero rows for a six-word query as something to explain, since a legitimately selective query can return nothing. And if your fusion scores look "flat", compute $1/(k + r + 1)$ before concluding anything about fusion. None of these signals is a verdict on effectiveness: on these collections a leg that was alive and overlapping made the hybrid worse than the dense retriever alone more often than better (significantly worse in five pairs, significantly better in two), and that question needs relevance judgements, not scores.

9 Limitations

The production measurements are on ten and fifteen queries over private corpora and are reported as aggregates the reader cannot check; they establish that the failure happened and how it looked, not an effect size, and the claim that the leg had been inert since the system's first version is an inference from unchanged code. The public reproduction uses six

collections and three encoders of one size class (768-dimensional, base-sized) and FTS5's default BM25 weighting of title and text; the three encoders rerun the same queries, so the 18 pairs are not 18 independent samples, and the paired tests are corrected within each family of 18 but not across the six families. The collections differ in several ways at once, so the orderings reported here (empty rate against content words per query, short-list disjointness across collections of different size) are observations over five or six points, not controlled effects, and the pooled row-count rates mix the two builders, whose short lists arise differently. The truncation sweep covers only the two smallest collections. The exact ceiling statement needs $K \leq k$, and the guarantee that an empty leg is never missed needs two retrievers; the common $K = 100, k = 60$ setting gets the one-sided direction and the direct overlap count instead, and it produced two ceilings with overlap present. The ceiling detects zero overlap; attributing it to an empty leg needs the per-leg row counts, and a leg that returns one useless document the other leg also returns, as on ArguAna, passes both the ceiling and a zero-row check. The negative results in Appendix A are negative on one system each.

10 Conclusion

With per-retriever depth $K \leq k$, a maximum reciprocal-rank-fusion score of $1/k$ over a result list means no document in it was returned by more than one retriever, and a system that fuses can read that from the scores it already computes. In the incident, the ceiling had been present on every query for as long as the keyword query builder had existed; it was read twice as a property of fusion. On six BEIR collections the ceiling missed no empty keyword leg in 22 362 runs, fired over a nonempty but disjoint leg on 174, mostly short lists on the larger corpora, and twice over lists that shared one document deep in both at $K > k$. It is blind by construction to a leg that returns a document the other leg also returns, which ArguAna shows with a leg that is never empty and almost never relevant; the distribution of row counts is the instrument for that case. Repairing the leg raised overlap in every pair and moved nDCG@10 above the dense retriever alone in 5 of 18, two of them significantly, and below it in 13, five significantly, so the health of a leg and the value of fusing it remain separate questions.

Use of AI tools

The code, the experiment and audit scripts, and the text of this paper were drafted with Claude (Anthropic) running in Claude Code, directed by the author, who operated the production system in which the incident occurred, chose what to measure, ruled on every claim and takes responsibility for all content. Drafts were reviewed by GPT-6 (Astra, Sol and Luna), GPT-6.1 Sol, SWE-2, Gemini 4 Argon, Gemini, Grok 4.7, DeepSeek V4.1 Flash, GPT-5.6 and Claude Fable 5.1, through Cognition's Devin CLI, Cursor, Kagi and vendor CLIs; every defect a reviewer reported was reproduced or refuted by a script before it changed the paper, and the review records, probes and the audit script that checks every number in the text against the result files are in the repository. Every reference was checked against a primary listing.

Acknowledgments and Disclosure of Funding

This work received no funding. The author declares no competing interests.

References

Anthropic. Introducing contextual retrieval. `https://www.anthropic.com/engineering/contextual-retrieval`, 2024. Accessed 2026-10-07.

Vera Boteva, Demian Gholipour Ghalandari, Artem Sokolov, and Stefan Riezler. A full-text learning to rank dataset for medical information retrieval. In *Advances in Information Retrieval: 38th European Conference on IR Research*, pages 716–722, 2016.

Sebastian Bruch, Siyu Gai, and Amir Ingber. An analysis of fusion functions for hybrid retrieval. *ACM Transactions on Information Systems*, 42(1):20:1–20:35, 2024.

Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel Weld. SPECTER: Document-level representation learning using citation-informed transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2270–2282, 2020.

Michael D. Cooper. Design considerations in instrumenting and monitoring Web-based information retrieval systems. *Journal of the American Society for Information Science*, 49(10):903–919, 1998.

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

Steve Cronen-Townsend, Yun Zhou, and W. Bruce Croft. Predicting query performance. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 299–306, 2002.

Edward A. Fox and Joseph A. Shaw. Combination of multiple searches. In *Proceedings of the Second Text REtrieval Conference (TREC-2)*, pages 243–252, 1994.

Luyu Gao, Zhuyun Dai, Tongfei Chen, Zhen Fan, Benjamin Van Durme, and Jamie Callan. Complement lexical retrieval model with semantic residual embeddings. In *Advances in Information Retrieval: 43rd European Conference on IR Research*, pages 146–160, 2021.

D. Richard Hipp et al. SQLite FTS5 extension. `https://www.sqlite.org/fts5.html`, 2026. Accessed 2026-10-07.

Chris Kamphuis, Arjen P. de Vries, Leonid Boytsov, and Jimmy Lin. Which BM25 do you mean? a large-scale reproducibility study of scoring variants. In *Advances in Information Retrieval: 42nd European Conference on IR Research*, pages 28–34, 2020.

Joon Ho Lee. Analyses of multiple evidence combination. In *Proceedings of the 20th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 267–276, 1997.

Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. Sparse, dense, and attentional representations for text retrieval. *Transactions of the Association for Computational Linguistics*, 9:329–345, 2021.

Sean MacAvaney, Andrew Yates, Sergey Feldman, Doug Downey, Arman Cohan, and Nazli Goharian. Simplified data wrangling with ir_datasets. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2429–2436, 2021.

Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. WWW'18 open challenge: Financial opinion mining and question answering. In *Companion Proceedings of The Web Conference 2018*, pages 1941–1942, 2018.

Mark Montague and Javed A. Aslam. Relevance score normalization for metasearch. In *Proceedings of the Tenth International Conference on Information and Knowledge Management*, pages 427–433, 2001.

Rodrigo Nogueira, Wei Yang, Jimmy Lin, and Kyunghyun Cho. Document expansion by query prediction. *arXiv preprint arXiv:1904.08375*, 2019.

Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder. *arXiv preprint arXiv:2402.01613*, 2024.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems, Datasets and Benchmarks Track*, 2021.

Christophe Van Gysel and Maarten de Rijke. Pytrec_eval: An extremely fast python interface to trec_eval. In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 873–876, 2018.

Ellen Voorhees, Tasmeer Alam, Steven Bedrick, Dina Demner-Fushman, William R. Hersh, Kyle Lo, Kirk Roberts, Ian Soboroff, and Lucy Lu Wang. TREC-COVID: Constructing a pandemic information retrieval test collection. *ACM SIGIR Forum*, 54(1):1–12, 2020.

Henning Wachsmuth, Shahbaz Syed, and Benno Stein. Retrieval of the best counterargument without prior topic knowledge. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 241–251, 2018.

David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 7534–7550, 2020.

Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

William Webber, Alistair Moffat, and Justin Zobel. A similarity measure for indefinite rankings. *ACM Transactions on Information Systems*, 28(4):20:1–20:38, 2010.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packed resources for general Chinese embeddings. *arXiv preprint arXiv:2309.07597*, 2023.

Appendix A. Two negative results measured beside the dead leg

Both of the following were measured on the production system before the keyword leg was fixed, with seven to fifteen queries each; both verdicts stand on their own terms, and neither says anything about the techniques in general.

Recency decay. A multiplicative age penalty on fused scores (floor 0.85, time constant 180 days) changed the top-1 result on 4 of 7 real queries on a notes corpus, and the changes were as often wrong as right; a canonical note unedited for 338 days was demoted under a 13-day-old draft. On a second corpus spanning only 63 days the multiplier varied between 0.956 and 1.000 across every document and changed 0 of 8 top-1 results. The feature was rejected. At the time the sensitivity was attributed to flat fused scores; Section 1 shows the relative step between consecutive singleton scores is a property of the formula, and two identical live lists would show the same step, so the flatness said nothing about the leg, and whether decay would behave differently over a fused list with real two-leg scores was not tested.

A title prefix before embedding. Prepending `kind: title` to each chunk before embedding (the stored text and the keyword index unchanged, so the vector was the only variable) and re-embedding one corpus of 3 046 documents and 30 734 chunks took 25 minutes. On 15 realistic queries, 13 had an identical top-1, mean top-5 overlap was 80%, one query improved and one regressed, and the query predicted to benefit most returned an identical, still wrong, top 3. The change was not adopted. This is a heading prefix: neither document expansion by generated queries (Nogueira et al., 2019) nor the model-written situating context of Anthropic (2024), and the corpus was a weak test (documents whose title is their opening line). Because the keyword leg was inert, the A/B measured the vector change alone, which was the intent.

Appendix B. All conditions, all collections

Tables 6, 7 and 8 give all six conditions for each encoder. The BM25-only rows do not depend on the encoder and repeat across the three tables. “kw empty” is the fraction of

queries on which the keyword leg returned no document; "at ceiling" the fraction on which the maximum fused score was $1/k$; "both-leg" the mean number of documents credited by both retrievers. Query times are in `results/*.json` and are omitted here because the collections were timed on two different machines (Appendix C).

Appendix C. Run details

The SciFact and NFCorpus runs were first made on CPU (12-core x86-64, PyTorch CPU wheels), about an hour per collection and encoder for document embedding. All 18 collection–encoder pairs were then run in a Kaggle kernel on one NVIDIA T4 (`kaggle/run.py` in the repository, which clones the runner at a pinned commit and records package versions and the device in a manifest): PyTorch 2.11 with CUDA 12.8, `transformers` 4.49, `sentence-transformers` 3.4.1, `ir_datasets` 0.6.3. At the same batch size, the GPU runs of the four pairs of the earlier version of this paper (SciFact and NFCorpus with bge and nomic) reproduced the CPU results to every digit the result files carry for nDCG@10 and Recall@100; every result file in the repository is from the Kaggle runs, and the CPU runs figure only as that reproduction check. Documents were embedded in batches of 64, except with nomic, which accepts 8 192 tokens where the other two encoders stop at 512 and ran out of GPU memory at that size on the three largest collections, so every nomic run reported here used batches of 8. Batch size reaches the vectors only through padding and the order of floating-point operations; between batch 64 and batch 8 the nomic runs on SciFact were identical and on NFCorpus one query's fused maximum score changed in the fourth decimal, with no printed figure affected. `nomic-embed-text-v1.5` needs a 4.x `transformers` because its remote model code calls a method removed in 5.0; the runner pins it below 4.50. The encoder's document prefix is applied where its model card prescribes one (`search_document:` for nomic, `passage:` for e5, none for bge); queries carry the query prefix (`search_query:` for nomic; `query:` for e5; `Represent this sentence for searching relevant passages:` for bge, which its card asks for on retrieval queries and on nothing else). The FTS5 index is built atomically. ArguAna, SciDocs and TREC-COVID have a single test split in BEIR and are loaded as `beir/arguana`, `beir/scidocs` and `beir/trec-covid`.

The fixed builder's stoplist, 118 words: *a an the and or but if then than that this these those of in on at to for from by with about into over under after before is are was were be been being am do does did doing have has had having i me my mine we us our you your he she it its they them their as not no nor so such can could should would will shall may might must what which who whom whose when where why how all any both each few more most other some only own same too very just dont don t s re ve ll m d get got make made want need know think see look.*

Per-query diagnostics are in `results/*.perquery.jsonl` and per-query effectiveness scores in `results/*.pereval.jsonl`; `scripts/keyword_facts.py` computes the encoder-free keyword facts, `scripts/tables.py`, `scripts/natural_lengths.py` and `scripts/figure.py` the tables, per-query counts and Figure 1, `scripts/significance.py` the paired tests, and the audit script `review/orrery-audit.py` checks every number in this paper against the result files.

Collection	Condition	nDCG@10	R@100	kw empty	at ceiling	both-leg
SciFact	bm25-and	0.018	0.018	98.0%	–	–
SciFact	bm25-or-raw	0.659	0.879	0.0%	–	–
SciFact	bm25-or	0.665	0.876	0.0%	–	–
SciFact	dense	0.740	0.967	–	–	–
SciFact	hybrid-and	0.741	0.967	98.0%	98.0%	0.02
SciFact	hybrid-or	0.723	0.969	0.0%	0.0%	28.9
NFCorpus	bm25-and	0.193	0.095	50.8%	–	–
NFCorpus	bm25-or-raw	0.304	0.232	7.7%	–	–
NFCorpus	bm25-or	0.306	0.233	7.7%	–	–
NFCorpus	dense	0.374	0.337	–	–	–
NFCorpus	hybrid-and	0.382	0.341	50.8%	51.7%	4.0
NFCorpus	hybrid-or	0.363	0.337	7.7%	8.7%	21.3
ArguAna	bm25-and	0.001	0.001	0.0%	–	–
ArguAna	bm25-or-raw	0.473	0.957	0.0%	–	–
ArguAna	bm25-or	0.484	0.965	0.0%	–	–
ArguAna	dense	0.641	0.993	–	–	–
ArguAna	hybrid-and	0.641	0.993	0.0%	0.0%	1.0
ArguAna	hybrid-or	0.606	0.991	0.0%	0.0%	38.6
SciDocs	bm25-and	0.003	0.002	94.2%	–	–
SciDocs	bm25-or-raw	0.152	0.343	0.0%	–	–
SciDocs	bm25-or	0.153	0.344	0.0%	–	–
SciDocs	dense	0.217	0.496	–	–	–
SciDocs	hybrid-and	0.216	0.495	94.2%	94.9%	0.13
SciDocs	hybrid-or	0.195	0.469	0.0%	0.1%	29.3
FiQA	bm25-and	0.014	0.015	81.8%	–	–
FiQA	bm25-or-raw	0.233	0.521	0.0%	–	–
FiQA	bm25-or	0.224	0.508	0.0%	–	–
FiQA	dense	0.406	0.741	–	–	–
FiQA	hybrid-and	0.396	0.740	81.8%	87.2%	0.44
FiQA	hybrid-or	0.348	0.715	0.0%	0.3%	22.1
TREC-COVID	bm25-and	0.075	0.003	74.0%	–	–
TREC-COVID	bm25-or-raw	0.532	0.091	0.0%	–	–
TREC-COVID	bm25-or	0.538	0.101	0.0%	–	–
TREC-COVID	dense	0.779	0.141	–	–	–
TREC-COVID	hybrid-and	0.744	0.140	74.0%	90.0%	0.86
TREC-COVID	hybrid-or	0.763	0.133	0.0%	0.0%	21.6

Table 6: `bge-base-en-v1.5`, all conditions.

Collection	Condition	nDCG@10	R@100	kw empty	at ceiling	both-leg
SciFact	bm25-and	0.018	0.018	98.0%	–	–
SciFact	bm25-or-raw	0.659	0.879	0.0%	–	–
SciFact	bm25-or	0.665	0.876	0.0%	–	–
SciFact	dense	0.703	0.928	–	–	–
SciFact	hybrid-and	0.705	0.928	98.0%	98.0%	0.02
SciFact	hybrid-or	0.708	0.958	0.0%	0.0%	25.9
NFCorpus	bm25-and	0.193	0.095	50.8%	–	–
NFCorpus	bm25-or-raw	0.304	0.232	7.7%	–	–
NFCorpus	bm25-or	0.306	0.233	7.7%	–	–
NFCorpus	dense	0.347	0.298	–	–	–
NFCorpus	hybrid-and	0.361	0.303	50.8%	51.7%	3.7
NFCorpus	hybrid-or	0.344	0.309	7.7%	8.4%	21.0
ArguAna	bm25-and	0.001	0.001	0.0%	–	–
ArguAna	bm25-or-raw	0.473	0.957	0.0%	–	–
ArguAna	bm25-or	0.484	0.965	0.0%	–	–
ArguAna	dense	0.483	0.980	–	–	–
ArguAna	hybrid-and	0.483	0.980	0.0%	0.0%	1.0
ArguAna	hybrid-or	0.517	0.984	0.0%	0.0%	43.7
SciDocs	bm25-and	0.003	0.002	94.2%	–	–
SciDocs	bm25-or-raw	0.152	0.343	0.0%	–	–
SciDocs	bm25-or	0.153	0.344	0.0%	–	–
SciDocs	dense	0.187	0.404	–	–	–
SciDocs	hybrid-and	0.187	0.404	94.2%	94.8%	0.17
SciDocs	hybrid-or	0.182	0.409	0.0%	0.1%	32.3
FiQA	bm25-and	0.014	0.015	81.8%	–	–
FiQA	bm25-or-raw	0.233	0.521	0.0%	–	–
FiQA	bm25-or	0.224	0.508	0.0%	–	–
FiQA	dense	0.376	0.682	–	–	–
FiQA	hybrid-and	0.367	0.682	81.8%	87.5%	0.45
FiQA	hybrid-or	0.330	0.675	0.0%	0.0%	24.1
TREC-COVID	bm25-and	0.075	0.003	74.0%	–	–
TREC-COVID	bm25-or-raw	0.532	0.091	0.0%	–	–
TREC-COVID	bm25-or	0.538	0.101	0.0%	–	–
TREC-COVID	dense	0.823	0.150	–	–	–
TREC-COVID	hybrid-and	0.784	0.148	74.0%	88.0%	0.70
TREC-COVID	hybrid-or	0.769	0.137	0.0%	0.0%	21.9

Table 7: `nomic-embed-text-v1.5`, all conditions.

Collection	Condition	nDCG@10	R@100	kw empty	at ceiling	both-leg
SciFact	bm25-and	0.018	0.018	98.0%	–	–
SciFact	bm25-or-raw	0.659	0.879	0.0%	–	–
SciFact	bm25-or	0.665	0.876	0.0%	–	–
SciFact	dense	0.719	0.963	–	–	–
SciFact	hybrid-and	0.722	0.963	98.0%	98.0%	0.02
SciFact	hybrid-or	0.726	0.959	0.0%	0.0%	27.9
NFCorpus	bm25-and	0.193	0.095	50.8%	–	–
NFCorpus	bm25-or-raw	0.304	0.232	7.7%	–	–
NFCorpus	bm25-or	0.306	0.233	7.7%	–	–
NFCorpus	dense	0.355	0.319	–	–	–
NFCorpus	hybrid-and	0.368	0.323	50.8%	51.4%	3.8
NFCorpus	hybrid-or	0.352	0.328	7.7%	7.7%	20.6
ArguAna	bm25-and	0.001	0.001	0.0%	–	–
ArguAna	bm25-or-raw	0.473	0.957	0.0%	–	–
ArguAna	bm25-or	0.484	0.965	0.0%	–	–
ArguAna	dense	0.447	0.949	–	–	–
ArguAna	hybrid-and	0.447	0.949	0.0%	0.0%	1.0
ArguAna	hybrid-or	0.532	0.981	0.0%	0.0%	33.5
SciDocs	bm25-and	0.003	0.002	94.2%	–	–
SciDocs	bm25-or-raw	0.152	0.343	0.0%	–	–
SciDocs	bm25-or	0.153	0.344	0.0%	–	–
SciDocs	dense	0.187	0.421	–	–	–
SciDocs	hybrid-and	0.186	0.421	94.2%	94.7%	0.15
SciDocs	hybrid-or	0.183	0.425	0.0%	0.1%	31.7
FiQA	bm25-and	0.014	0.015	81.8%	–	–
FiQA	bm25-or-raw	0.233	0.521	0.0%	–	–
FiQA	bm25-or	0.224	0.508	0.0%	–	–
FiQA	dense	0.399	0.732	–	–	–
FiQA	hybrid-and	0.389	0.732	81.8%	88.4%	0.36
FiQA	hybrid-or	0.362	0.711	0.0%	0.3%	19.0
TREC-COVID	bm25-and	0.075	0.003	74.0%	–	–
TREC-COVID	bm25-or-raw	0.532	0.091	0.0%	–	–
TREC-COVID	bm25-or	0.538	0.101	0.0%	–	–
TREC-COVID	dense	0.696	0.128	–	–	–
TREC-COVID	hybrid-and	0.669	0.127	74.0%	90.0%	0.78
TREC-COVID	hybrid-or	0.714	0.124	0.0%	0.0%	19.6

Table 8: `e5-base-v2`, all conditions.